

LAB 6 – GREATEST COMMON DIVISOR

The goal of this lab was to create a circuit that computed the GCD of two 8-bit numbers. The GCD calculator part in this design selects the 8-bit inputs using two multiplexers. It also makes use of three registers to load the inputs and output. There is also a control module that regulates the design. All these modules are used to create a top module

Procedure

The GCD module was written as the first step in this lab. The module takes two 8-bit numbers and subtracts the big one from the small one until the big one is less than the other, then reverses the numbers and subtracts again. This provides the result. Two multiplexers are used to select which inputs to load into registers. Three registers are used: two for inputs and one for output. Then, to control the design, a control module is created. After developing a control module and a GCD module, all modules are combined into a top GCD module.

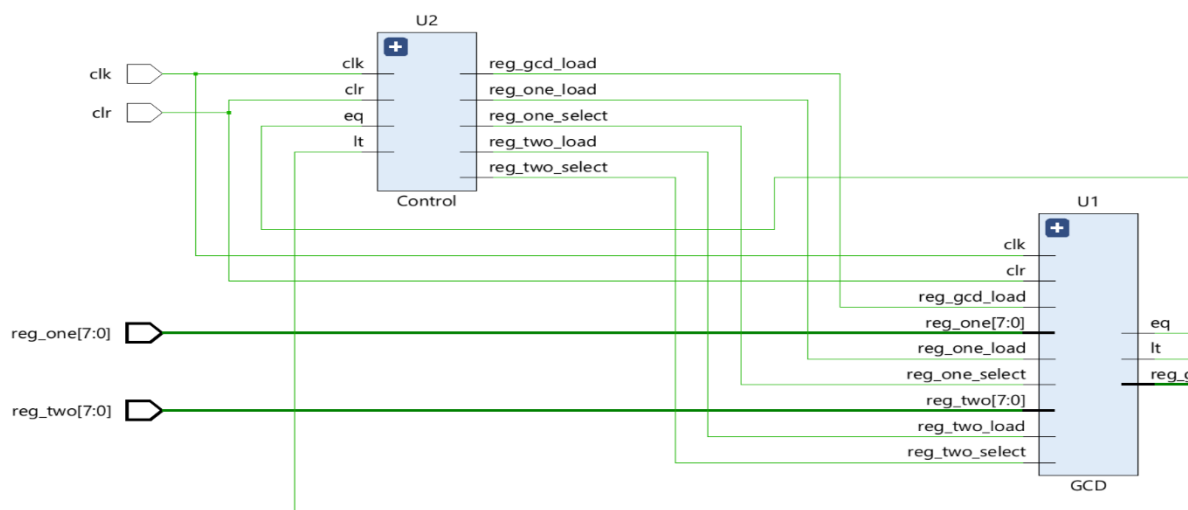


Figure 1: Schematics of the design

Conclusion

Question 1) What was your algorithm to calculate the GCD?

This design's algorithm subtracts the second 8-bit input from the first 8-bit input until the first input is less than the second input. The algorithm then swaps the two inputs, subtracts one more time, and returns the result.

Question 2) Is your module a combinational circuit or an FSM? If the latter, is it a Moore machine or a Mealy machine? Would it be cheaper to implement GCD as a combinational circuit or as an FSM? Would it be faster? What are the drawbacks in each case?

This design is a Moore machine. Using a Moore machine saves money because combinational circuits require fewer flip flops than logic gates. FSM is slower than a combinational circuit. The disadvantages of using a combinational circuit are that, while it is faster, it is more expensive, and if

one component in a series circuit fails, the entire circuit fails. The disadvantage of using a Moore machine is that it is slower than a combinational circuit but less expensive.

Question 3) How many clock cycles did it take in your simulation to calculate the GCD? Do you think this can be optimized? How so?

The simulation took 40 clock cycles to calculate the GCD. By adding more states and writing a more detailed architecture, this operation can be made faster.

Observations

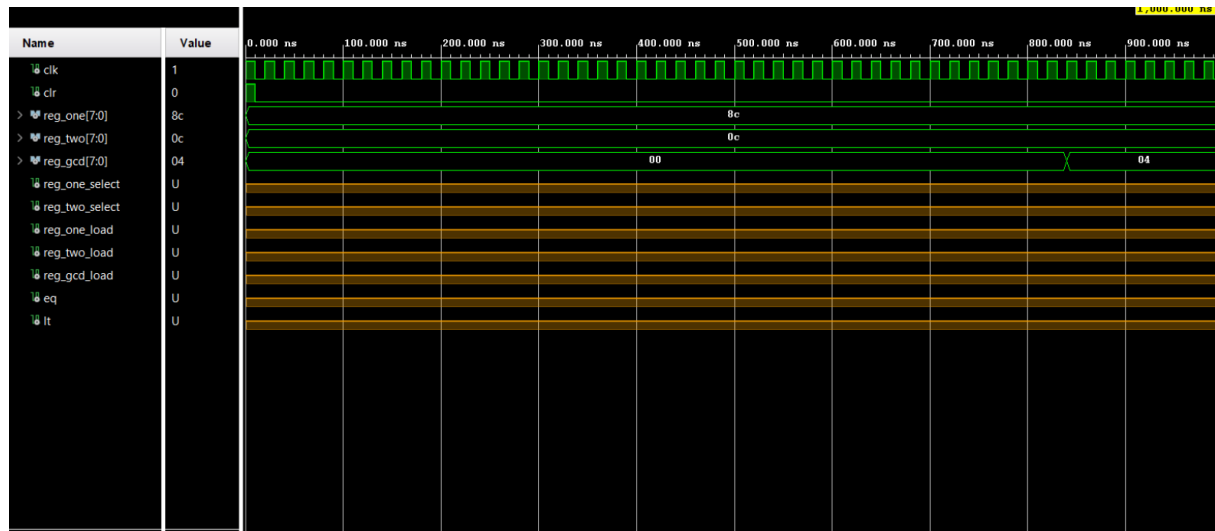


Figure 2: Testbench

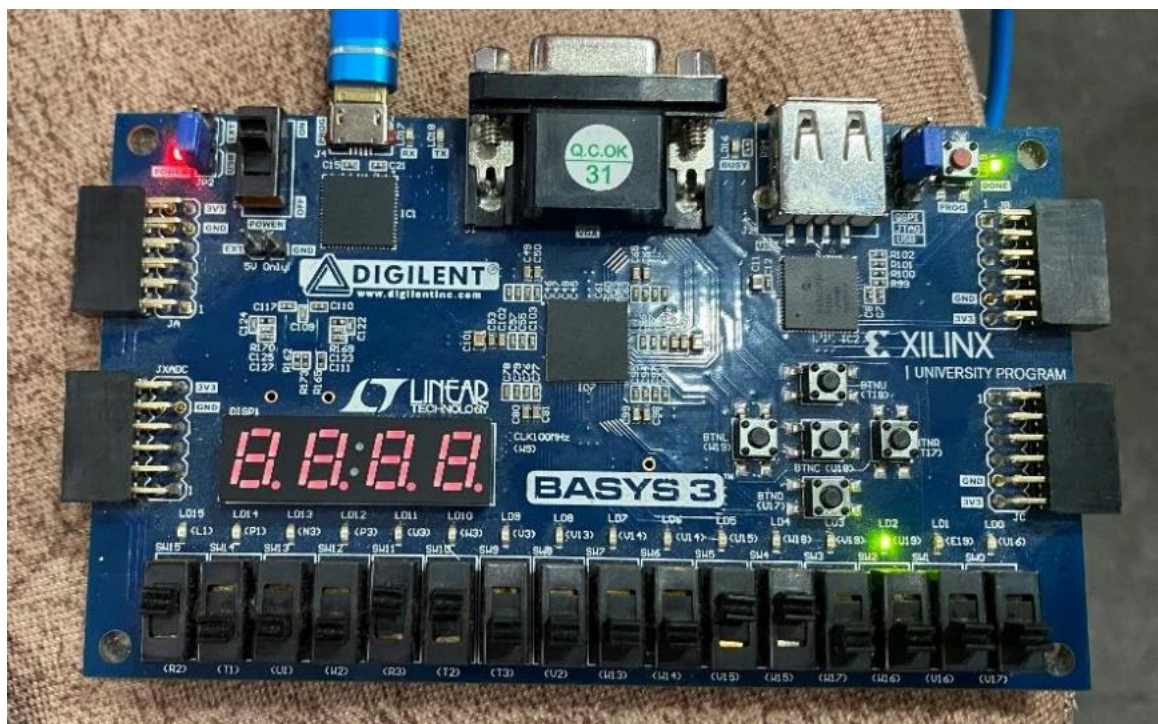


Figure 3: BASYS 3 Board [right side is 48 (00110000), left side is 140 (10001100), the result is 4(100)]

Hande Eryılmaz
21902678

Codes

TOP MODULE

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity TOP_MODULE is
Port ( clk: in std_logic;
clr: in std_logic;
reg_one : in STD_LOGIC_VECTOR (7 downto 0);
reg_two : in STD_LOGIC_VECTOR (7 downto 0);
reg_gcd: out STD_LOGIC_VECTOR (7 downto 0));
end TOP_MODULE;
```

```
architecture Behavioral of TOP_MODULE is
component GCD
Port ( clk: in std_logic;
clr: in std_logic;
reg_one : in STD_LOGIC_VECTOR (7 downto 0);
reg_two : in STD_LOGIC_VECTOR (7 downto 0);
reg_select1: in std_logic;
reg_select2: in std_logic;
reg_load1: in std_logic;
reg_load2: in std_logic;
reg_load_gcd: in std_logic;
var2: out std_logic;
var1: out std_logic;
reg_gcd: out STD_LOGIC_VECTOR (7 downto 0));
end component;
```

```
component Control
Port ( clk: in std_logic;
clr: in std_logic;
var1: in std_logic;
var2: in std_logic;
reg_select1: out std_logic;
reg_select2: out std_logic;
reg_load1: out std_logic;
reg_load2: out std_logic;
reg_load_gcd: out std_logic);
end component;
```

```
signal reg_select1: std_logic;
signal reg_select2: std_logic;
signal reg_load1: std_logic;
signal reg_load2: std_logic;
signal reg_load_gcd: std_logic;
signal var1: std_logic;
signal var2: std_logic;
begin
```

```
U1: GCD port map(clk=> clk, clr=> clr, reg_one => reg_one,
reg_two => reg_two, reg_select1 => reg_select1,
reg_select2 => reg_select2, reg_load1 => reg_load1, reg_load2
=> reg_load2, reg_load_gcd => reg_load_gcd, var2=> var2,
var1=> var1, reg_gcd => reg_gcd);
```

```
U2: Control port map (clk => clk, clr => clr, var2=> var2, var1=>
var1, reg_select1 => reg_select1, reg_select2 => reg_select2,
reg_load1 => reg_load1, reg_load2 => reg_load2, reg_load_gcd
=> reg_load_gcd);
end Behavioral;
```

GCD

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity GCD is
Port ( clk: in std_logic;
clr: in std_logic;
reg_one : in STD_LOGIC_VECTOR (7 downto 0);
reg_two : in STD_LOGIC_VECTOR (7 downto 0);
reg_select1: in std_logic;
reg_select2: in std_logic;
reg_load1: in std_logic;
reg_load2: in std_logic;
reg_load_gcd: in std_logic;
var2: out std_logic;
var1: out std_logic;
reg_gcd: out STD_LOGIC_VECTOR (7 downto 0));
end GCD;
```

```
architecture Behavioral of GCD is
component mux
Port (
a :in std_logic_vector (7 downto 0);
b: in std_logic_vector (7 downto 0);
s: in std_logic;
y: out std_logic_vector (7 downto 0));
end component;
```

```
component reg_file
Port (
load: in std_logic;
D: in std_logic_vector (7 downto 0);
clk: in std_logic;
clr: in std_logic;
Q: out std_logic_vector (7 downto 0));
end component;
```

```
signal number_one: std_logic_vector (7 downto 0);
signal number_two: std_logic_vector (7 downto 0);
signal number1_new: std_logic_vector (7 downto 0);
signal number2_new: std_logic_vector (7 downto 0);
signal onetotwo: std_logic_vector (7 downto 0);
signal twotoone: std_logic_vector (7 downto 0);
```

```
begin
onetotwo <= number_one - number_two;
twotoone <= number_two - number_one;
```

```
process (number_one, number_two) -- Var1uality flag
begin
if number_one = number_two then var1 <= '1';
else var1 <= '0';
end if;
end process;
```

```
process (number_one, number_two)-- less than flag
begin
if number_one < number_two then var2 <= '1';
else var2 <= '0';
end if;
end process;
```

```
M1: mux
port map (a => onetotwo, b => reg_one, s => reg_select1, y =>
number1_new);
M2: mux
port map (a => twotoone, b => reg_two, s => reg_select2, y =>
number2_new);
```

Hande Eryılmaz

21902678

```
Reg1: reg_file
port map (load => reg_load1, d => number1_new ,clk => clk ,clr
=> clr ,q => number_one);
Reg2: reg_file
port map (load => reg_load2, d => number2_new ,clk => clk ,clr
=> clr ,q => number_two);
Reg3: reg_file
port map (load => reg_load_gcd, d => number_one ,clk => clk ,clr
=> clr ,q => reg_gcd);
end Behavioral;
```

Control Unit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity Control is
Port ( clk: in std_logic;
      clr: in std_logic;
      var1: in std_logic;
      var2: in std_logic;
      reg_select1: out std_logic;
      reg_select2: out std_logic;
      reg_load1: out std_logic;
      reg_load2: out std_logic;
      reg_load_gcd: out std_logic);
end Control;
```

```
architecture Behavioral of Control is
type state_type is (input, test1, test2, update1, update2, done);
signal present_state, next_state: state_type;
```

```
begin
process (clk, clr) --state register
begin
    if clr ='1' then present_state <= input;
    elsif rising_edge (clk)then present_state <=
        next_state;
    end if;
end process;
```

```
process (present_state, var1, var2) -- input process
begin
case present_state is
when input => next_state <= test1;
when test1 =>
    if var1 = '1' then next_state <= done;
    else next_state <= test2;
    end if;
when test2 =>
    if var2 = '1' then next_state <= update1;
    else next_state <= update2;
    end if;
when update1 => next_state <= test1;
when update2 => next_state <= test1;
when done => next_state <= done;
when others => null;
end case;
end process;
```

```
process (present_state) -- output process
begin
reg_select1 <= '0';
reg_select2<= '0';
reg_load1 <= '0';
reg_load2 <= '0';
reg_load_gcd <= '0';
case present_state is
when input =>
    reg_select1 <= '1';
    reg_select2<= '1';
```

```
    reg_load1 <= '1';
    reg_load2 <= '1';
when update1 => reg_load2 <= '1';
when update2 => reg_load1 <= '1';
when done => reg_load_gcd <= '1';
when others => null;
end case;
end process;
end Behavioral;
```

Multiplexer

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity mux is
Port (
a :in std_logic_vector (7 downto 0);
b: in std_logic_vector (7 downto 0);
s: in std_logic;
y: out std_logic_vector (7 downto 0));
end mux;
```

```
architecture Behavioral of mux is
begin
process (a,b,s)
begin
    if s ='0' then y <= a;
    else y <= b;
    end if;
end process;
end Behavioral;
```

Register

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity reg_file is
Port (
load: in std_logic;
D: in std_logic_vector (7 downto 0);
clk: in std_logic;
clr: in std_logic;
Q: out std_logic_vector (7 downto 0));
end reg_file;
```

```
architecture Behavioral of reg_file is
begin
process (clk, clr)
begin
    if clr ='1' then Q <= (others => '0');
    elsif rising_edge (clk) then
        if load = '1' then
            Q <= D;
        end if;
    end if;
end process;
end Behavioral;
```

Hande Eryılmaz

21902678

Constraint

```
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0
5} [get_ports clk]
set_property PACKAGE_PIN U18 [get_ports clr]
set_property IOSTANDARD LVCMOS33 [get_ports clr]
set_property PACKAGE_PIN W19 [get_ports start_signal]
set_property IOSTANDARD LVCMOS33 [get_ports start_signal]
set_property PACKAGE_PIN V17 [get_ports {reg_one[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_one[0]}]
set_property PACKAGE_PIN V16 [get_ports {reg_one[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_one[1]}]
set_property PACKAGE_PIN W16 [get_ports {reg_one[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_one[2]}]
set_property PACKAGE_PIN W17 [get_ports {reg_one[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_one[3]}]
set_property PACKAGE_PIN W15 [get_ports {reg_one[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_one[4]}]
set_property PACKAGE_PIN V15 [get_ports {reg_one[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_one[5]}]
set_property PACKAGE_PIN W14 [get_ports {reg_one[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_one[6]}]
set_property PACKAGE_PIN W13 [get_ports {reg_one[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_one[7]}]
set_property PACKAGE_PIN V2 [get_ports {reg_two[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_two[0]}]
set_property PACKAGE_PIN T3 [get_ports {reg_two[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_two[1]}]
set_property PACKAGE_PIN T2 [get_ports {reg_two[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_two[2]}]
set_property PACKAGE_PIN R3 [get_ports {reg_two[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_two[3]}]
set_property PACKAGE_PIN W2 [get_ports {reg_two[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_two[4]}]
set_property PACKAGE_PIN U1 [get_ports {reg_two[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_two[5]}]
set_property PACKAGE_PIN T1 [get_ports {reg_two[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_two[6]}]
set_property PACKAGE_PIN R2 [get_ports {reg_two[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_two[7]}]
# LEDs
set_property PACKAGE_PIN U16 [get_ports {reg_gcd[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_gcd[0]}]
set_property PACKAGE_PIN E19 [get_ports {reg_gcd[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_gcd[1]}]
set_property PACKAGE_PIN U19 [get_ports {reg_gcd[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_gcd[2]}]
set_property PACKAGE_PIN V19 [get_ports {reg_gcd[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_gcd[3]}]
set_property PACKAGE_PIN W18 [get_ports {reg_gcd[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_gcd[4]}]
set_property PACKAGE_PIN U15 [get_ports {reg_gcd[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_gcd[5]}]
set_property PACKAGE_PIN U14 [get_ports {reg_gcd[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_gcd[6]}]
set_property PACKAGE_PIN V14 [get_ports {reg_gcd[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg_gcd[7]}]
```

Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity testbench is
end testbench;
architecture Behavioral of testbench is
COMPONENT GCD_TOP
Port ( clk: in std_logic;
clr: in std_logic;
reg_one : in STD_LOGIC_VECTOR (7 downto 0);
reg_two : in STD_LOGIC_VECTOR (7 downto 0);
reg_gcd: out STD_LOGIC_VECTOR (7 downto 0));
end component;

signal clk: std_logic:= '1';
signal clr: std_logic:= '1';
signal reg_one : STD_LOGIC_VECTOR (7 downto 0):=
"00000000";
signal reg_two : STD_LOGIC_VECTOR (7 downto 0):=
"00000000";
signal reg_gcd: STD_LOGIC_VECTOR (7 downto 0);
signal reg_select1: std_logic;
signal reg_select2: std_logic;
signal reg_load1: std_logic;
signal reg_load2: std_logic;
signal reg_load_gcd: std_logic;
signal eq: std_logic;
signal lt: std_logic;
begin
clk <= not clk after 10 ns;
UUT: GCD_TOP port map (
    clk => clk,
    clr => clr,
    reg_one => reg_one,
    reg_two => reg_two,
    reg_gcd => reg_gcd);
stim_proc: process
begin
reg_one <= "10001100";
reg_two <= "00001100";
wait for 10ns;
clr <= '0';
wait;
end process;
end Behavioral
```